

Product Requirement Document: Dual-Sided Restaurant Table Reservation and Floor Operations Platform

Architectural Foundation and Platform Paradigm

The modern food and beverage hospitality sector requires continuous digital synchronization between front-of-house table inventory and consumer demand channels¹. Enterprise dining ecosystems, popularized by platforms such as OpenTable, EazyDiner, and Zomato District, resolve this coordination problem by deploying dual-sided systems: an administrative console that controls table inventory, dining shifts, and kitchen ticket pacing, and a consumer discovery portal that facilitates real-time reservation scheduling¹.

This document defines the functional and technical specifications for a Proof of Concept (POC) designed to validate core transactional integrity, seating state transitions, and real-time order tracking². To maintain development velocity and isolate architectural risk during early-stage testing, the platform intentionally circumvents external infrastructure bottlenecks³. Automated device-level geolocation querying is replaced with fixed neighborhood directory filtering, third-party payment processing is deferred in favor of informational discovery indices, and variable dining durations are standardized into a deterministic thirty-minute table turnaround window³.

The presentation tier decouples restaurant operational controls from consumer interactions while binding both to a unified relational state engine². The administrative dashboard acts as a live operations terminal, giving floor managers visual command over table occupancy, seating allocations, and kitchen fulfillment timers².

Concurrently, the consumer application functions as a discovery and booking funnel, allowing diners to select a target locality, inspect venue commercial benchmarks, verify table availability, and submit booking requests¹. Beneath the interface layer, the platform executes transactional row locking to resolve concurrent booking attempts, preventing table double-allocation while coordinating asynchronous status notifications².

Architectural Domain	Operational Mandate	POC Implementation Pattern	Production Target Architecture
Merchant Administration	Venue operating hours, seating	Direct web-based configuration forms	Role-based access control with

	layout, and availability status management ³ .	with reactive state updates ⁹ .	multi-unit hierarchy and audit logging ⁹ .
Floor State Engine	Real-time monitoring of table occupancy, order tickets, and service timers ² .	Tabular grid dashboard with client-side interval synchronization ² .	Bi-directional WebSocket vector canvas with distributed publish-subscribe sync ² .
Consumer Booking	Geographic discovery, inventory querying, and reservation submission ¹ .	Static administrative locality dropdown with restaurant profile cards ³ .	Reverse-geocoded spatial search engine with algorithmic yield pricing ³ .
Concurrency & Integrity	Prevention of race conditions and conflicting reservation assignments ³ .	Relational database row-level locking over discrete thirty-minute windows ² .	Distributed Redis key leases with automatic time-to-live expiration ³ .

Merchant Floor Operations and Administrative Specifications

Operating Schedules and Shift Management

The administrative terminal provides venue managers with direct authority over daily temporal boundaries, ensuring the booking engine accepts customer traffic only during active operational windows⁴. The system models time using standardized fifteen-minute operational intervals⁴. Administrators input a formal opening timestamp (T_{open}) and closing timestamp (T_{close}) for standard service shifts⁴.

The scheduling validation engine automatically rejects booking submissions where the requested arrival time precedes the shift opening or where the service completion window exceeds the terminal closing threshold⁴. Furthermore, floor supervisors have access to a global operational toggle that switches the venue status between active booking acceptance and blackout mode, enabling immediate floor closure during kitchen disruptions, severe weather, or private buyouts without altering underlying schedule parameters⁹.

Inventory Topology and Table Structural Constraints

Physical seating capacity is structured into discrete, addressable table assets rather than an

abstract aggregate venue headcount³. This granular structure allows the reservation engine to map incoming party sizes directly to physical furniture limitations, maintaining efficient dining room utilization⁴.

Each table record maintains an explicit alphanumeric identifier, an assigned dining room zone,

and strict seating boundary thresholds defined by a minimum capacity ($C_{\min}$) and a

maximum capacity ($C_{\max}$)⁴. For the POC deployment, inventory is organized into two

standardized operational models: two-seater tables configured with $C_{\min} = 1$ and

$C_{\max} = 2$, and four-seater tables configured with $C_{\min} = 3$ and $C_{\max} = 4$ ⁴.

Administrators can add, alter, or remove table assets dynamically through the console,

modifying total floor capacity⁴. The system prevents the deletion or structural downsizing of

any table associated with active or future confirmed reservations, protecting existing bookings from accidental data corruption⁴.

Deterministic Turnaround Timing Engine

Table turnover calculations govern reservation slot availability and operational pacing⁶. In mature enterprise environments, table dwell times fluctuate dynamically based on guest count, course counts, and culinary formats, ranging from forty-five to ninety minutes⁶.

To eliminate algorithmic complexity and provide stable scheduling during the initial trial phase, the system enforces a globally hardcoded table turnaround duration of exactly thirty minutes:

$$\Delta t_{\text{TAT}} = 30 \text{ minutes}$$

When an administrative user seats a party or the system confirms an automated booking

starting at timestamp T_{start} , the inventory engine marks that physical unit unavailable until the arrival of the release threshold:

$$T_{\text{release}} = T_{\text{start}} + \Delta t_{\text{TAT}}$$

Any subsequent reservation inquiry requesting an interval overlapping

$[T_{\text{start}}, T_{\text{start}} + 30 \text{ min})$ is identified as a scheduling conflict and omitted from customer availability results⁴. The administrative settings interface exposes this turnaround parameter as a visible configuration element, establishing the technical foundation for future releases to transition from hardcoded constants to variable, party-size-dependent durations⁶.

Real-Time Floor State Monitoring

The floor dashboard presents a visual layout of dining room operations, allowing hosts to track seating availability, active covers, and service turnover³. Each physical table entity is rendered as a distinct operational tile reflecting real-time state changes driven by customer arrivals, order entries, or host adjustments².

Seating State	Visual Status Indicator	State Transition Trigger	Operational Implication & Enforcement
Available	High-Visibility Green	System initialization, shift reset, or completed guest checkout ² .	Unit is vacant and eligible for immediate walk-in seating or automated reservation allocation ⁹ .
Reserved	Controlled Amber	Host approval of a pending consumer booking or manual staff reservation hold ² .	Unit is locked for a specific thirty-minute block; excluded from customer search results ³ .
Occupied	Operational Blue	Host seats an arriving party against a booking or records a walk-in party ³ .	Unit is occupied; links active food order tickets and starts the table service timer ² .
Closed / Blocked	Neutral Slate Gray	Manual administrative intervention taking the table out of service ⁴ .	Unit is removed from available floor inventory due to maintenance or staffing constraints ⁴ .

In-Service Order Tracking and Live Elapsed Timers

The administrative interface integrates floor management directly with kitchen operations, giving staff visibility into ticket delivery timelines and dining room pacing². When a table transitions to the occupied state, the console associates an active food and beverage order ticket with that specific table record².

The order tracking module captures the placement timestamp (t_{placed}) when the order is initially entered, followed by the acknowledgment timestamp (t_{received}) when the kitchen display acknowledges ticket preparation⁴. The ticket view displays the itemized list of ordered

dishes, unit quantities, custom preparation notes, and the running total charge¹⁰. Simultaneously, the table tile initiates an active, client-side ticking elapsed timer that computes total elapsed dining duration in real time:

$$\Delta t_{\text{elapsed}} = t_{\text{current}} - t_{\text{placed}}$$

This live counter runs continuously while the table remains occupied². To support timely table turnover, the visual indicator incorporates an operational alert threshold⁶. If $\Delta t_{\text{elapsed}}$ reaches or exceeds the thirty-minute mark ($\Delta t_{\text{elapsed}} \geq 30 \text{ min}$), the timer display shifts from standard neutral typography to a flashing amber or red alert⁶. This visual cue alerts floor hosts that the dining party has exceeded the standard service turnaround window, prompting staff to present the bill or prepare the table for incoming reservations⁶.

Host Reservation Management and Floor Overrides

Front-of-house hospitality requires balancing automated reservations with unpredictable in-person guest arrivals³. The administrative console functions as a supervisory control center, giving hosts full authority over the reservation lifecycle⁴.

When a customer submits a booking through the consumer portal, the request enters an administrative triage queue marked with the status Requested⁴. Hosts review party sizes, target times, and customer notes before clicking an approval control to confirm the booking or a rejection control to decline it⁴.

In addition to managing digital requests, hosts can manually reserve tables for phone-in patrons or walk-in parties by selecting an available table, inputting guest details, and instantly confirming the seating⁴.

The console also allows staff to revoke reservations or clear active table holds when guests fail to arrive within a designated grace period, returning the table asset to the available inventory pool³.

Consumer Discovery and Booking Portal Specifications

Static Locality Discovery Funnel

To prioritize transactional booking flows and avoid third-party spatial API dependencies, billing complexities, and browser permission hurdles, the consumer interface uses a deterministic location selection workflow³.

A persistent dropdown menu in the global application header displays pre-configured dining districts, commercial sectors, and municipal zones (e.g., "Cyber City, Gurgaon", "Koramangala, Bangalore", "Bandra Kurla Complex, Mumbai")³. Selecting a locality filters the restaurant directory, presenting only venues whose operational addresses belong to the chosen geographic sector³. This approach gives users predictable control over neighborhood selection without relying on device GPS coordinates, keeping the prototype focused on inventory

reservation workflows³.

Restaurant Directory Presentation and Discovery Benchmarks

The discovery feed organizes participating dining venues into structured directory cards modeled after industry benchmarks like Zomato District and EazyDiner¹. These cards display core operational and pricing indicators to help consumers choose an appropriate venue¹. Each card displays the restaurant name, high-resolution visual branding, culinary categories, aggregated user review score, and current operational status¹. To set clear price expectations, cards prominently display two standard economic indicators: the approximate spend for an individual diner (Approximate Cost for One) and the standardized couple benchmark (Approximate Cost for Two)³. These values are derived from average menu selections, allowing diners to assess price levels before entering the booking flow³.

Reservation Booking Funnel

Clicking a restaurant card directs the user to an inventory-aware reservation booking interface¹. Diners select their target dining date, party size, and arrival time via a dynamic fifteen-minute slot selector⁴.

The consumer interface sends an availability query to the server, verifying that the restaurant has an unreserved table matching the requested party size throughout the specified thirty-minute window⁴. If the inventory check succeeds, the user completes the booking form by providing their full name, phone number, and optional dietary requests⁴. Submitting the form generates a pending reservation record with the initial status Requested, holding the table asset while routing the booking to the restaurant console for review⁴.

Transactional Status Updates and Notification Pipeline

Because table confirmation requires host verification, the consumer application provides clear visual status updates throughout the booking process⁹.

Upon form submission, the consumer interface displays a holding screen indicating that the request has reached the venue and is awaiting host approval⁹. When the restaurant host accepts the booking in the administrative console, the backend updates the reservation record to Confirmed and triggers a confirmation event³. In the POC build, this confirmation updates the on-screen badge to Confirmed and logs an asynchronous mock SMS or WhatsApp payload to the system console (e.g., "Reservation Confirmed: Table for 2 at Restaurant ABC on [Date] at [Time]")³. This mock notification system provides an audit trail for verification messaging while keeping the architecture ready for live SMS gateway integration³.

Consumer View Component	Interactive Elements	Underlying Data Payloads	Functional Validation Rules
Locality Header Selector	Administrative locality dropdown selector ³ .	Target locality_id, sector geographic string ³ .	Selection persists across view transitions; filters

			restaurant listings ³ .
Venue Discovery Card	Clickable restaurant profile card with cost metrics ¹ .	Venue name, cuisine categories, ratings, cost_for_one, cost_for_two ¹ .	Hides venues flagged as globally inactive or outside the active locality ⁴ .
Slot Booking Form	Date picker, time slot selector (15-min intervals), party size selector (1 to 4 guests) ⁴ .	restaurant_id, date, start_time, party_size, guest_name, guest_phone ⁴ .	Blocks times outside operating hours and party sizes lacking matching tables ⁴ .
Booking Tracker View	Live reservation status banner, confirmation modal, cancellation trigger ⁹ .	reservation_id, current status (Requested, Confirmed, Cancelled) ⁴ .	Polls or listens for merchant approval; confirms table assignment upon acceptance ³ .

Data Architecture and Entity-Relationship Definitions

The operational platform uses a normalized relational schema to enforce ACID transaction guarantees, eliminate orphaned booking records, and maintain referential integrity under concurrent booking attempts³.

Primary domain entities are structured hierarchically¹⁴. A single Restaurant entity maintains a one-to-many relationship with its physical Table inventory and daily operational shift definitions¹⁴. Each Customer entity connects to multiple historical and active Reservation records¹⁶.

A Reservation links exactly one Customer to a specific Table asset within a designated Restaurant for a bounded thirty-minute interval⁴. When guests are seated, an active Reservation maps to a single Order entity, which maintains a one-to-many relationship with itemized OrderItem records representing kitchen tickets⁷.

Entity Model	Primary Key	Foreign Key References	Core Attributes & Storage Types	Operational Role & Business Rules
Restaurant	restaurant_id (UUID)	None	name (VARCHAR),	Defines the physical

			locality (VARCHAR), opening_time (TIME), closing_time (TIME), cost_for_one (NUMERIC), cost_for_two (NUMERIC), is_active (BOOLEAN) ¹⁴ .	operating establishment, geographic boundary, operational hours, and discovery pricing benchmarks ³ .
Table	table_id (UUID)	restaurant_id	table_number (VARCHAR), min_capacity (INT), max_capacity (INT), current_status (ENUM: Available, Reserved, Occupied, Closed) ⁴ .	Represents the addressable seating units within a specific venue, enforcing structural party size constraints ⁴ .
Customer	customer_id (UUID)	None	full_name (VARCHAR), phone_number (VARCHAR), email (VARCHAR), created_at (TIMESTAMP) ⁴ .	Stores identifying patron credentials required for reservation verification and notification dispatch ³ .
Reservation	reservation_id (UUID)	restaurant_id, table_id, customer_id	party_size (INT), reservation_date (DATE), start_time	Canonical transaction ledger recording claims over

			(TIME), end_time (TIME), status (ENUM: Requested, Confirmed, Cancelled, Seated, Completed) ⁴ .	table inventory during specific thirty-minute time intervals ³ .
Order	order_id (UUID)	table_id, reservation_id	order_status (ENUM: Placed, Received, Preparing, Served, Billed), placed_at (TIMESTAMP), received_at (TIMESTAMP), total_amount (NUMERIC) ⁷ .	Tracks live dining room commerce associated with a physical table during an active seating session ² .
OrderItem	order_item_id (UUID)	order_id	item_name (VARCHAR), quantity (INT), unit_price (NUMERIC), notes (TEXT) ¹⁰ .	Granular line items detailing kitchen fulfillment requirements and itemized ticket billing ¹⁰ .

Concurrency Control, Algorithmic Allocation, and Lifecycle Transitions

Concurrency Race Mitigation

In digital reservation systems, high-concurrency race conditions occur when multiple patrons attempt to book the final available table for the same time slot simultaneously³. Unchecked concurrent requests can result in double bookings, creating operational friction and harming the customer experience².

To prevent conflicting allocations without introducing complex distributed lock managers during the POC phase, the system enforces concurrency control directly at the relational database level².

When a booking request passes party-size validation, the backend opens a database

transaction that acquires an exclusive row-level lock on candidate table records for the requested restaurant and service window². The system queries the reservation ledger to verify whether an overlapping booking exists with an active status (Requested, Confirmed, or Seated)³.

If no overlapping booking is detected, the transaction inserts the new reservation with status Requested and commits, automatically releasing the row lock². If a concurrent thread attempts to evaluate the same table for an overlapping time slot, its transaction blocks until the first completes².

Upon acquiring the lock, the second transaction detects the newly inserted reservation, identifies the schedule conflict, rolls back, and returns an HTTP 409 Conflict error⁴. This mechanism guarantees that table inventory is never double-allocated³.

Mathematical Pacing and Overlap Formulations

Reservation pacing rules ensure smooth guest throughput during operating shifts⁴. The operational capacity of any physical table is defined by the shift opening time T_{open} , the shift closing time T_{close} , and the standardized thirty-minute turnaround constant Δt_{TAT} ⁴.

The maximum number of non-overlapping booking intervals (N_{slots}) supported by a single table during a normal shift is calculated as:

$$N_{\text{slots}} = \left\lfloor \frac{T_{\text{close}} - T_{\text{open}}}{\Delta t_{\text{TAT}}} \right\rfloor$$

For any proposed reservation request starting at timestamp T_i , the system defines the terminal reservation boundary as:

$$T_{\text{end}} = T_i + 30 \text{ minutes}$$

To evaluate scheduling viability, candidate tables are checked for conflicting bookings within the interval $[T_{\text{start}}, T_{\text{end}})$ ⁴. A schedule conflict between an existing reservation interval $R_{\text{existing}} = [S_e, E_e)$ and a newly requested reservation interval $R_{\text{new}} = [S_n, E_n)$ exists if and only if:

$$\max(S_e, S_n) < \min(E_e, E_n)$$

If this condition evaluates to true for any active reservation tied to the table, the table is marked conflicted and excluded from the candidate availability list for that interval⁴.

Best-Fit Table Allocation Strategy

When a consumer requests a reservation without selecting a specific table, the booking engine applies a best-fit allocation algorithm to assign the most appropriate table while preserving

capacity for larger parties⁴:

The algorithm queries all tables associated with the venue that have no overlapping bookings during $[T_{\text{start}}, T_{\text{start}} + 30 \text{ min})$ and whose status is Available⁴. It filters this set to tables

where the guest count P satisfies $C_{\min} \leq P \leq C_{\max}$ ⁴.

Candidate tables are then sorted in ascending order of seat differential:

$$\Delta C = C_{\max} - P$$

The table with the lowest non-negative ΔC is selected for the reservation⁴. Under this rule, a party of two ($P = 2$) will match a two-seater table ($C_{\max} = 2, \Delta C = 0$) rather than a four-seater table ($C_{\max} = 4, \Delta C = 2$), reserving larger tables for groups of three or four diners⁴.

If all two-seater tables are occupied, the algorithm can fall back to four-seater tables ($\Delta C = 2$) if configured to prioritize booking completion over capacity optimization⁴.

Reservation and Order State Machine Transitions

Both reservations and active dining orders follow structured, deterministic state machines to synchronize customer expectations with kitchen and floor operations⁷.

Domain Process	Originating State	Target Transition State	Operational Event & Condition	System Action & Interface Updates
Reservation	<i>Initial Submission</i>	REQUESTED	Customer submits date, time, and party size matching available inventory ⁴ .	Inserts pending reservation record; displays ticket in merchant triage queue ⁴ .
Reservation	REQUESTED	CONFIRMED	Restaurant host clicks approval action in management console ⁹ .	Sets table state to Reserved; triggers customer confirmation

				notification toast ³ .
Reservation	REQUESTED	CANCELLED	Host rejects booking or customer aborts request prior to confirmation ⁴ .	Marks reservation cancelled; releases temporary inventory hold ⁴ .
Reservation	CONFIRMED	SEATED	Host marks party as arrived within operational grace period ³ .	Changes table state to Occupied; initializes order ticket and elapsed timer ² .
Reservation	CONFIRMED	CANCELLED	Host cancels hold due to guest no-show past arrival grace period ¹³ .	Reverts table state to Available; logs cancellation in audit history ² .
Reservation	SEATED	COMPLETED	Host settles order billing and marks table vacated ¹⁰ .	Returns table state to Available; stops elapsed timer; archives session ² .
Kitchen Order	<i>Order Creation</i>	PLACED	Server or patron submits food and beverage selections ⁴ .	Sets t_{placed} timestamp; starts kitchen elapsed timer on table card ² .
Kitchen Order	PLACED	RECEIVED	Kitchen display or floor manager acknowledges incoming	Writes t_{received} timestamp; updates ticket

			ticket ⁷ .	status in active order drawer ⁷ .
Kitchen Order	RECEIVED	SERVED	Food expediter confirms all line items delivered to table ⁷ .	Changes ticket badge to served; maintains elapsed dwell timer ² .
Kitchen Order	SERVED	BILLED	Host generates folio and registers payment transaction ¹⁰ .	Closes active ticket; unlocks reservation completion transition ² .

Technical Specifications and Implementation Boundaries

Controlled Functional Boundaries

To validate core product assumptions efficiently, several enterprise platform features are replaced with controlled, lightweight implementations during the POC phase³.

Operational Feature	Proof of Concept Implementation	Production Target Architecture
Table Turnaround Model	Hardcoded thirty-minute window ($\Delta t_{\text{TAT}} =$) applied across all party sizes ⁶ .	Dynamic machine-learning yield engine that adjusts turnaround based on party size, cuisine, and historical pacing ⁵ .
Geographic Discovery	Static administrative locality dropdown selector in header ³ .	Geocoded geospatial indexing with GPS positioning and dynamic radius searches ³ .
Notification Pipeline	In-app visual confirmation toasts and mock asynchronous logging	Direct integration with transactional SMS and WhatsApp Business

	payloads ³ .	gateways ³ .
Floor Plan Layout	Structured tabular dashboard rendering table status tiles ⁹ .	Interactive 2D drag-and-drop vector canvas matching physical restaurant floor layouts ³ .
Payment & Settlement	Displays informational pricing metrics (cost_for_one, cost_for_two) without payment collection ³ .	Integrated payment gateway (e.g., Stripe, Razorpay) handling deposits and no-show fees ³ .

Performance Metrics and Verification Thresholds

The platform must satisfy explicit performance thresholds during POC testing to verify system stability before moving toward production deployment:

The availability search endpoint must return candidate table payloads within an execution latency of:

$$t_{\text{response}} \leq 250 \text{ ms}$$

under simulated single-venue loads of up to fifty concurrent requests⁴. Concurrency controls must achieve a zero-tolerance standard for scheduling anomalies: under simultaneous conflicting booking simulations, the transactional locking engine must guarantee zero double-booking occurrences³.

Client-side elapsed order timers must synchronize against server timestamps upon every ticket lifecycle change, maintaining temporal drift below:

$$\Delta t_{\text{drift}} \leq 1.0 \text{ second}$$

throughout the thirty-minute dining window².

Phased Implementation Roadmap

Developing a reliable dual-sided table management platform requires proving core transactional stability before introducing operational complexity³. Hardcoding the service turnaround duration to thirty minutes establishes a predictable baseline for testing floor pacing, host approval workflows, and database locking without the variance of dynamic dining durations³.

Similarly, replacing automated device geolocation with fixed administrative neighborhoods keeps early engineering focused on data integrity and booking workflows rather than third-party map integrations³.

Development should advance across three structured phases:

In Phase 1 (Transactional Core Validation), engineering teams establish the relational database

schema, implement database row-level locking to prevent conflicting bookings, and build the administrative dashboard with table status tiles, host acceptance queues, and live elapsed timers².

In Phase 2 (Operational Synchronization), the platform transitions from client polling to bidirectional WebSocket channels for real-time timer updates, integrates transactional SMS and WhatsApp gateways for automated customer notifications, and expands the turnaround model to accommodate party-size-dependent durations (e.g., forty-five minutes for two-seaters, seventy-five minutes for four-seaters)³.

In Phase 3 (Enterprise Scale Deployment), the architecture replaces database row locks with distributed Redis key leases for high-concurrency surge handling, introduces interactive 2D canvas floor plan editors, and integrates real-time payment processing for automated deposits and no-show fee protection³.

Focusing the initial build on core operational requirements—clear table capacities, reliable transactional locking, and direct host approval workflows—creates a stable, validated technical foundation for scaling into a comprehensive restaurant management platform³.

Works cited

1. Restaurant Reservation Management Overview | PDF | Point Of Sale, <https://www.scribd.com/document/752974557/BRD-Restaurant>
2. TableTaps | Restaurant SaaS Platform - Ashish Kumar, <https://decoderashish.com/projects/tabletaps/>
3. Build Your Own OpenTable Alternative | RapidDev, <https://www.rapidevelopers.com/clone/opentable>
4. woki-lite/woki-lite.md at main · serugeneris/woki-lite - GitHub, <https://github.com/serugeneris/woki-lite/blob/main/woki-lite.md>
5. Eatigo Restaurant Booking App | 7,000+ Partners | Ortem, <https://ortemtech.com/case-studies/eatigo-restaurant-reservations/>
6. AI-Powered Restaurant Table Turnover Optimization For Efficient, <https://www.myshyft.com/blog/restaurant-table-turnover-analysis/>
7. User Manual for Food Space Deluxe System | PDF - Scribd, <https://www.scribd.com/document/516112019/AP04-AA5-EV05-Ingles-Elaboracion-de-un-manual-tecnico-convertido>
8. Table Reservation System | PDF - Slideshare, <https://www.slideshare.net/slideshow/table-reservation-system/81448773>
9. Best WordPress Restaurant Booking Plugin 2026: 7 Options, <https://javistab.com/wordpress-restaurant-booking-plugin-7options-compared/>
10. 9700 Man SQL PDF | PDF | Databases | Portable Document Format, <https://www.scribd.com/document/373643957/60522104-9700-MAN-SQL-pdf>
11. Restaurant Table Turnover Rate: formula and strategies - Qamarero, <https://qamarero.com/en/blog/restaurant-table-turnover-rate-formula-and-strategies/>
12. Reorder Card Feature Overview | PDF | Menu - Scribd, <https://www.scribd.com/document/693326507/Reorder-Reorder-Card>

13. Modeling no-shows, cancellations, overbooking, and walk-ins in,
https://www.researchgate.net/publication/304611250_Modeling_no-shows_cancellations_overbooking_and_walk-ins_in_restaurant_revenue_management
14. System design of Restaurant Table reservation system - Medium,
<https://medium.com/@aayushi.tongia/system-design-of-restaurant-table-reservation-system-5795569bfd9d>
15. Apache Iceberg ACID Transactions for Data Lakehouses - IOMETE,
<https://iomete.com/resources/blog/apache-iceberg-acid-transactions-catalog>
16. Restaurant Management System ERD Overview | PDF - Scribd,
<https://www.scribd.com/document/868249070/erd-rms>
17. Java's Synchronization Toolkit: The Locking Mechanism - Medium,
<https://medium.com/@amitvsolutions/javas-synchronization-toolkit-the-locking-mechanism-882fbbe57921>
18. 9700 Man SQL | PDF | File Format | Databases - Scribd,
<https://www.scribd.com/document/60522104/9700-MAN-SQL>